

Devine Café: uso das redes de Petri em um sistema de máquina de café

Allyson Bruno de Freitas Fernandes
UFERSA

Pau dos Ferros, Brasil
allyson.fernandes@alunos.ufersa.edu.br

Andrey de Oliveira Sabino
UFERSA

Pau dos Ferros, Brasil
andrey.sabino@alunos.ufersa.edu.br

Geísa Moraes Gabriel
UFERSA

Pau dos Ferros, Brasil
geisa.gabriel@alunos.ufersa.edu.br

Klebson Davi de Souza Magalhães
UFERSA

Pau dos Ferros, Brasil
klebson.magalhaes@alunos.ufersa.edu.br

Lívia Beatriz Maia de Lima
UFERSA

Pau dos Ferros, Brasil
livia.lima30332@alunos.ufersa.edu.br

Pedro Damião de Oliveira Luz
UFERSA

Pau dos Ferros, Brasil
pedro.luz@ufersa.edu.br

Resumo—Este artigo focaliza o processo de desenvolvimento de um sistema por meio do uso de métodos formais de engenharia de software utilizando redes de Petri. A aplicação desse modelo matemático incide sobre um sistema de máquina de café intitulado Devine Café, criado visto a necessidade de tornar o processo de pedidos em uma cafeteria menos complexo e mais ágil. Para tal, pretende-se evidenciar a relevância da aplicação de processos formais e também de técnicas de qualidade de software no referido sistema. Com isso, procura-se entender mais detalhadamente a abordagem utilizada para cada um dos mecanismos supracitados, assim como retratar a relevância de construir sistemas conforme os aspectos de produção determinados pela engenharia de software. O presente trabalho justifica-se por relacionar aspectos de apoio ao desenvolvimento de sistemas visando reduzir erros, melhorar a comunicação e facilitar a manutenção por meio da integração e automatização dos processos. Assim, este artigo representa uma pesquisa de tipologia teórica a partir da criação de documentos, manuais e relatórios sobre o Devine Café, assim como da perspectiva prática do desenvolvimento de software em linguagem de programação C#. Revela-se, por meio deste, a pertinência em aplicar estratégias de aprimoramento de software com o uso de conceitos formais para a modelagem, análise e verificação de sistemas. Sobre tal conjuntura, a aplicação das redes de Petri para o Devine Café garantiu maior precisão, confiabilidade e segurança em relação às funcionalidades atribuídas ao sistema.

Index Terms—Sistema de Máquina de Café, Métodos Formais, Redes de Petri, Qualidade de Software.

I. INTRODUÇÃO

As interações humano-máquina compreendem um processo realizado em etapas que podem ser divididas em vários níveis, dependendo do perfil do usuário [16]. A realização de pedidos em uma cafeteria, por exemplo, pode acarretar alguns problemas, como erros de comunicação provenientes de pedidos mal interpretados, atrasos no atendimento e limitações de personalização de pedidos com precisão. Logo, em um sistema de máquina de café, facilitar a realização de pedidos em uma cafeteria, por meio da automatização de processos, promove maior autonomia ao cliente e, por consequência, permite gerar maiores lucros em um cenário comercial.

A fim de justificar a viabilidade em volta da motivação para realizar o desenvolvimento do sistema, foi efetuada uma breve pesquisa voltada, principalmente, ao consumo do produto e à frequência dos consumidores em cafeterias. Nesse sentido, 29% dos brasileiros tomam mais de seis xícaras de café por dia, enquanto 46% preferem de três a cinco xícaras [1]. Além disso, o fomento em volta da criação de um sistema de máquina de café justifica-se pela alta na frequência em cafeterias (51% em 2023) após a pandemia da Covid-19, causada pela possibilidade de interagir com as pessoas, qualidade e modo de preparo do café [1].

Visto a viabilidade do produto de software e tendo em vista o desenvolvimento de sistemas, é necessário que o programa seja usável e compreensível por humanos, bem como atue de forma confiável e funcione corretamente [21].

Pensando no desenvolvimento do sistema de máquina de café conforme a aplicação dos métodos formais, foi desenvolvido neste estudo o Devine Café. Tendo em vista a ocorrência de falhas humanas no gerenciamento de pedidos, realizou-se a automatização de processos conforme o uso de uma linguagem formal baseada em redes de Petri - ferramentas matemáticas gráficas que oferecem um ambiente propício para modelagem, análise e simulação de sistemas de eventos discretos [6].

Logo, a aplicação dos métodos formais para o Devine Café ocorre visando auxiliar na melhor compreensão dos requisitos e na identificação precoce de inconsistências, a fim de minimizar a complexidade e assegurar a correta implementação do sistema. Por essa razão, no contexto da engenharia de software, os métodos formais têm sido um dos meios utilizados para aumentar a qualidade e produtividade de software [17].

II. FUNDAMENTAÇÃO TEÓRICA

Em um contexto onde a tecnologia está cada vez mais presente, cresce a busca por sistemas computacionais de qualidade, com bom desempenho, baixo custo e com poucas falhas [7]. Tais características motivam a aplicabilidade de métodos para avaliar esses sistemas [8]. A partir do desenvolvimento

de um sistema Web de pedidos para uma cafeteria, aplica-se a definição de métodos formais de software por meio da utilização de redes de Petri.

Um sistema Web é, normalmente, constituído por um conjunto de páginas Web e por uma base de dados [11]. Esses sistemas utilizam a arquitetura cliente-servidor. Neles, a comunicação do cliente no navegador ocorre através do endereço IP (*Internet Protocol*) do servidor, responsável por hospedar a estrutura de dados [14].

Os sistemas Web permitem que os usuários acessem serviços, recursos e informações por meio de interfaces disponibilizadas pelos navegadores Web. O acesso e troca de informações ocorre utilizando protocolos de comunicação (HTTP/HTTPS) baseados em texto que permitem a transferência de informações entre clientes e servidores web na internet [12]. O sistema Web de máquina de café, Devine Café, propõe facilitar pedidos em uma cafeteria por meio da automatização de processos. Para isso, aplicam-se métodos formais de software com redes de Petri.

Nesse processo, a aplicação de métodos formais de software consiste em abordagens rigorosas para desenvolver e verificar sistemas, utilizando modelos matemáticos para descrever e analisar suas funcionalidades e comportamento. Com isso, é possível garantir que programas estão corretos ou apresentam certas propriedades [10]. Logo, os métodos formais podem ser utilizados como suporte à geração de bons conjuntos de casos de teste e até mesmo para orientar no encontro de soluções mais elegantes, simples e eficientes [10].

Entre os métodos de modelagem formal existentes, podem-se destacar as redes de Petri. Rede de Petri é um formalismo matemático representado graficamente e usado para modelar e analisar sistemas que tenham atividades paralelas, concorrentes, assíncronas e não-determinísticas [8]. Os lugares e transições das redes de Petri são usados para modelar uma visão lógica de pontos dos sistemas [20].

Aliado à clara relação entre métodos formais e excelência atribuída aos softwares, o emprego de testes também é relevante para a descoberta de inconsistências e validação dos requisitos de um sistema de software [21]. Para tal, cria-se uma gama de condições que descrevem os comportamentos a serem testados no software a partir da análise das especificações do sistema, denominados casos de teste [18]. Dessa forma, o teste é a principal técnica utilizada para a qualidade do sistema, uma vez que contribui na avaliação, controle e confiança do software [21].

Para aplicação dos métodos formais, bem como para o desenvolvimento de sistemas em sua totalidade, a documentação de software é essencial por dois motivos principais, uma das razões é facilitar a comunicação no processo de desenvolvimento do projeto e outra para esclarecer o conhecimento do programa nas atividades de manutenção [9]. A documentação atua, portanto, como uma facilitadora, esclarecendo precisamente o comportamento esperado do programa, bem como os possíveis desvios de fluxo [15]. Para isso, a especificação formal consiste na utilização de notações formais, baseadas em técnicas matemáticas e na lógica formal, para a especificação

de sistemas, permitindo reduzir erros e ambiguidades [4].

Além dos documentos, é comum adotar ferramentas a fim de auxiliar o ciclo de vida do software para facilitar seu gerenciamento e execução, bem como para seguir prazos e a utilização de recursos [3]. Para tanto, o PIPE2¹ é uma ferramenta de código aberto para criar e analisar redes Petri, possui análise avançada, tais como na simulação, que indica o número médio de *tokens* por lugar assim resultando informações mais completas de como a rede de Petri se comporta [13].

Para a realização dos testes, o Lighthouse² é uma ferramenta automatizada de código aberto incorporada ao DevTools³, criada para melhorar a qualidade das páginas da Web. Por meio da execução dos testes usando a ferramenta, é possível gerar um relatório sobre o desempenho da página web. O xUnit.net⁴, por sua vez, é uma ferramenta de teste de unidade, de código aberto, licenciado sob Apache 2. Os testes realizados pelo xUnit baseiam-se na comparação do resultado obtido ao se executar uma função com o resultado esperado [19].

Para a análise estática, o SonarQube IDE⁵ é um *plugin* para IDE gratuito e de código aberto responsável por encontrar e corrigir problemas de codificação. O SonarQube Cloud, por sua vez, consiste em uma ferramenta baseada em nuvem para fluxos de trabalho de Integração Contínua e Entrega/Implantação Contínua (CI/CD), que agiliza o trabalho dos desenvolvedores, permitindo testar quaisquer alterações feitas no repositório e fazer a entrega das alterações em diversos ambientes (produção ou testes) baseado em regras definidas, sem ser necessário utilizar serviços ou ferramentas externas [5].

O uso do SonarQube também garante a qualidade do código e aumenta a produtividade na resolução de problemas. Isso ocorre devido à tecnologia possui suporte para mais de 20 linguagens e usa mais de 5000 regras de *Clean Code* - conjunto de boas práticas de programação que buscam produzir código de fácil leitura, manutenção e evolução [2] - específicas de linguagem que buscam identificar erros comuns de codificação, *bugs* e vulnerabilidades.

III. ABORDAGEM

O presente estudo adotou uma abordagem de natureza aplicada ao desenvolvimento de um sistema de máquina de café utilizando Redes de Petri. Para tal, os resultados obtidos no alicerce da metodologia empregada são detalhados sobre a documentação criada para o sistema, incluindo a especificação formal, bem como na aplicação prática do desenvolvimento do software.

O processo de desenvolvimento do sistema ocorreu de forma iterativa e incremental, inicialmente, criou-se a base para o desenvolvimento back-end e front-end do projeto. Também foi criada a identidade visual e a padronização de informações do

¹Disponível em: <https://pipe2.sourceforge.net/>

²Disponível em: <https://developer.chrome.com/docs/lighthouse/overview?hl=pt-br>

³Disponível em: <https://developer.chrome.com/docs/devtools?hl=pt-br>

⁴Disponível em: <https://xunit.net/?tabs=cs>

⁵Disponível em: <https://www.sonarsource.com/products/sonarqube/>

sistema, com o Manual de Identidade Visual e Prototipagem do Software, assim como o Termo de Abertura do Projeto (TAP).

Posteriormente, criou-se a documentação do sistema para registrar as informações que caracterizam o software em sua totalidade. A partir disso, foram elicitados os requisitos funcionais e não funcionais, definiram-se as regras de negócio e foi realizada a modelagem do sistema por meio de diagramas de caso de uso e de classe, e também foi realizada a modelagem do banco de dados.

Para o desenvolvimento back-end utilizou-se a linguagem de programação C# e para o front-end a linguagem de programação TypeScript em conjunto com a biblioteca React, diante de uma arquitetura cliente-servidor. Para a persistência dos dados, foi utilizado o banco de dados Postgress em conjunto com o Docker. A API foi desenvolvida utilizando a arquitetura MVC, com uma implementação que segue o princípio da separação de responsabilidades. A estrutura do back-end determina-se com separações do código em camadas distintas para garantir a coesão do sistema, sendo elas: *repositories*, *services*, *validators*, *exceptions*, *resources*, *models*, *communication*, *data*, *migrations* e *persistence*.

A camada *repositories* (interfaces) é responsável pelo acesso aos dados. Já a camada *services* concentra a lógica de negócio da aplicação, enquanto a *validators* realiza a validação dos parâmetros recebidos do front-end. O tratamento de erros de forma customizada, conforme as regras do projeto, ocorre na camada *exceptions* e os arquivos contendo mensagens de erro, permitindo a internacionalização do sistema em diferentes idiomas, estão na camada *resources*. A *models* inclui as entidades, enums e uma classe base (Common) que serve como herança para as demais. A camada *communication* contém as classes de request e response utilizadas na comunicação com o front-end. Já a *data* é responsável pela configuração do Entity Framework, incluindo *migrations* das classes que serão persistidas no banco de dados; e a camada de *persistence*, onde são aplicadas as migrations e o seed dos dados.

A estruturação do front-end do sistema foi realizada por meio da biblioteca React com TypeScript. O roteamento da aplicação foi implementado com a biblioteca react-router-dom, permitindo navegação entre páginas distintas do sistema Web. As rotas implementadas foram: / (Home), /pedido (FacaPedido), /adicionais (Adicionais), /carrinho (Carrinho), /pagamento (TipoPagamento), /pedidofinalizado (PedidoFinalizado), /feedback (Feedback) e /cancelado (PedidoCancelado).

A rota /feedback foi posteriormente ajustada para /feedback/:cafeId, permitindo o envio do cafeId via URL. Para extrair o cafeId e repassar ao componente Feedback, foi criado o componente FeedbackPageRoute, utilizando o hook useParams.

No gerenciamento de Estado, para o Estado Local, foi utilizado o useState de forma extensiva para o controle do número de estrelas e observações no Feedback, para o Estado de seleção de xícaras no CafeCard e para as reações relacionadas ao atendimento. No Estado Global (Carrinho), fez-se o uso de CartProvider baseado em Context API, permitindo

que diversos componentes compartilhem e atualizem o estado centralizado do carrinho.

No que concerne aos componentes de UI e estilização, foram utilizados componentes reutilizáveis, como o CafeCard, para a exibição de cafés, e o Feedback, para o formulário de avaliação. A estilização e o material UI (MUI) ocorreu utilizando a biblioteca @mui/material, com destaque para o Rating (seleção de estrelas). Os ícones foram implantados com base na integração com o react-icons para exibição de ícones como FaRegStar, LuClock2 e IoCartSharp.

O componente CafeCard também é responsável pela lógica do negócio e interatividade, uma vez que permite selecionar o tamanho da xícara, com atualização automática de preço e estado de seleção. No CafeCard, foi implementada também a lógica para filtrar avaliações por café e calcular a média de estrelas, exibindo-a diretamente no card. Já o componente Feedback coleta estrelas, observações e reações, valida a entrada e envia os dados para o back-end.

Tratando-se dos endpoints implementados, o *postFeedback* envia dados de avaliação para o endpoint /api/avaliacoesCafe (ou /api/avaliacoes). Já o *getAvaliacaoCafe* busca avaliações existentes no backend para exibição nos CafeCards.

As chamadas de API envolvidas em blocos try... catch, com exibição de alerta ao usuário e logging no console em caso de falha, estão relacionadas à exibição do tratamento de erros par ao usuário final.

A modelagem com Redes de Petri aplicada sobre o Devine Café foi direcionada à garantia de corretude do fluxo, análise de propriedades como segurança e ausência de deadlock, e representação clara de comportamentos alternativos como cancelamento, alteração de pedido, visualização de detalhes e controle de tempo.

Foi utilizado o editor gráfico PIPE para modelar, simular e exportar a Rede de Petri criada. Para escrever a especificação formal completa, com conjuntos de lugares, transições, função de fluxo, pesos, marcação inicial e análise de propriedades, fez-se uso do editor de texto do Google (Google Docs).

Assim, as etapas para a construção da rede iniciaram com a análise dos requisitos funcionais e regras de negócio especificados no início do projeto por meio da documentação criada para o sistema. Essa etapa inclui a seleção de café, leite, açúcar e ingredientes; exibição e cálculo de valor; confirmação ou cancelamento do pedido; fluxos alternativos e exceções; e controle de tempo e personalizações.

Logo após, foram definidos os lugares (estados do sistema). Cada etapa lógica do pedido foi representada por um lugar (P), o que representaria um estado do sistema naquele determinado tempo de execução, funcionando com uma espécie de captura de tela. Exemplos disso foram o início do processo, o tipo de café escolhido, o pedido confirmado e o pedido entregue.

Na terceira etapa, ocorreu a definição das transições, representada pelas ações do usuário e do sistema, conforme os requisitos previamente definidos. Nas conexões por arcos (F), por sua vez, foram definidos todos os pares ordenados que conectam as ações aos estados. Cada arco representa o fluxo entre lugares e transições.

A marcação inicial define onde o token começa. No início do processo, essa marcação é igual a 1 e nos demais lugares inicia com zero. Para a modelagem também foram incluídos fluxos alternativos, além do fluxo principal (pedido → personalização → confirmação → preparo → entrega). Esses fluxos incluídos referem-se a alguns comportamentos que o sistema pode oferecer a partir de determinadas interações. São eles: fluxo de cancelamento com tempo limite, alteração do pedido antes da confirmação, detecção de item indisponível e visualização dos detalhes do pedido.

Por mim, efetuou-se a revalidação dos requisitos. Para isso, cada requisito funcional e regra de negócio foi cruzado com os elementos da rede. Os requisitos não aplicáveis à rede (como criptografia ou logs) foram documentados como fora do escopo de Petri, o que deve ser tratado em outra parte da documentação. Validou-se também a conformidade da rede no que diz respeito a aspectos de alcance, a rede permite alcançar todos os estados finais esperados; vivacidade, todas as transições são ativáveis em seus respectivos contextos; segurança, nenhum lugar pode conter múltiplos tokens simultaneamente; e, ausência de deadlock, o fluxo sempre segue até um estado final ou de erro.

IV. CONSIDERAÇÕES FINAIS E TRABALHOS FUTUROS

Este trabalho esteve estruturado sobre a necessidade de implementar modelos formais sobre softwares. Para isso, foi realizado o desenvolvimento de um sistema de máquina de café. Assim, concluiu-se que a aplicação de métodos formais direcionado para os processos de interações com usuário, escolha de bebidas e personalização de preferências garantem maior precisão, confiabilidade e segurança no funcionamento, uma vez que permitem identificar inconsistências no design do sistema antes que elas se manifestem no uso real.

O uso de ferramentas e o entendimento documentado sobre as funcionalidades do sistema também são formas de agilizar o processo de desenvolvimento e garantir uma melhor comunicação entre as partes envolvidas no projeto.

Como trabalhos futuros, espera-se aprimorar o sistema para a implementação em uma máquina de café real. Para isso, pretende-se aprimorar alguns artefatos criados, como os requisitos existentes, adicionar os requisitos futuros sugeridos e atualizar a especificação formal. Além disso, para o melhor funcionamento, espera-se refinar o sistema a partir da aplicação de mais técnicas de teste.

REFERÊNCIAS

- [1] ABIC. Indicadores da indústria de café, 2024.
- [2] AZEVEDO, B. A. A. S. Clean code e solid na construção da aplicação oratio: melhorando a manutenibilidade e escalabilidade.
- [3] BARBOSA, H. O., BONIFACIO, B. A., MENEZES, T. M., UEBEL, L. F., PIRES, F. B., AND NETO, A. F. Uma análise do uso de ferramentas em desenvolvimento distribuído de software para atualização da plataforma android. *WWW/INTERNET 2019* (2019), 39–46.
- [4] BARBOSA, R. D. M. Especificação formal de organizações de sistemas multiagentes.
- [5] BORGES, J. M. C. Pipeline ci/cd para automação e orquestração de redes.
- [6] CARGNIN, R. S., AND ROOS-FRANTZ, F. Revisão da literatura de simulação de sistemas de eventos discretos com foco na integração de aplicações. *III SFCT* (2015), 15.
- [7] DA COSTA AGUIAR, J. B., AND GRACIANO, F. C. Gestão da qualidade no desenvolvimento de produto de software. *Revista Interface Tecnológica* 18, 2 (2021), 773–783.
- [8] DE LIMA, J. W. S., FALCÃO, T. P., AND ANDRADE, E. Tryrdp: uma ferramenta para o aprendizado de modelagem de sistemas usando redes de petri. In *Simpósio Brasileiro de Educação em Computação (EDUCOMP)* (2021), SBC, pp. 362–370.
- [9] DE SOUZA, S. C. B., DAS NEVES, W. C. G., ANQUETIL, N., AND DE OLIVEIRA, K. M. Documentação essencial para manutenção de software ii. In *IV Workshop de Manutenção de Software Moderna (WMSWM), Porto de Galinhas, PE* (2007).
- [10] DÉHARBE, D., MOREIRA, A. M., RIBEIRO, L., AND RODRIGUES, V. M. Introdução a métodos formais: Especificação, semântica e verificação de sistemas concorrentes. *Revista de Informatica Teorica e Aplicada. Porto Alegre. Vol. 7, n. 1 (set. 2000), p. 7-48* (2000).
- [11] DELAMARO, M. E., MALDONADO, J. C., AND JINO, M. *Introdução ao Teste de Software*. Elsevier, 2016.
- [12] FIELDING, R. T., GETTYS, J., MOGUL, J., FRYSTYK, H., MASINTER, L., LEACH, P., AND BERNERS-LEE, T. Hypertext transfer protocol – http/1.1. Tech. Rep. RFC 2616, Internet Engineering Task Force (IETF), June 1999. Obsoleted by RFC 7230–7235.
- [13] KREBS, D. É. Estudo comparativo das ferramentas pipe2, mercury tool e timenet baseadas em redes de petri. *IV SFCT* 7 (2016), 5.
- [14] MARTHA, L. F. *Desenvolvimento de uma aplicação web para modelagem colaborativa*. PhD thesis, PUC-Rio, 2022.
- [15] MCCONNELL, S. *Code Complete: A Practical Handbook of Software Construction*, 2 ed. Microsoft Press, Redmond, WA, 2004.
- [16] NIELSEN, J. *Usability Engineering*. Morgan Kaufmann, San Francisco, 1993.
- [17] PIMENTA, A. Especificação formal de uma ferramenta de reutilização de especificações de requisitos.
- [18] PRESSMAN, R. S. *Engenharia de software*, 7 ed. AMGH, Porto Alegre, 2011.
- [19] SANTIN, C. E. Desenvolvimento guiado por testes e ferramentas xunit.
- [20] SILVA, A. N. D., LINS, F. A. A., SANTOS JÚNIOR, J. C., ROSA, N. S., QUENTAL, N. C., AND MACIEL, P. R. M. Avaliação de desempenho da composição de web services usando redes de petri.
- [21] SOMMERVILLE, I. *Engenharia de software*. Pearson Universidades, 2011.